

Lab 1: JAVA Multithread Programming

- Our course assumes all students have knowledge and experience of JAVA programming. Nevertheless, if you do not have the JAVA experience, you may want to watch following youtube video for obtaining basics of using JAVA.
- For JAVA code compilation (Eclipse install and setup):
 - <https://www.youtube.com/watch?v=rSFraftR4I4&t=663s>
- For JAVA beginner programming:
 - <https://youtu.be/U8wrZRYAnml>
- JAVA Programming Tutorial (6 hours)
 - https://youtu.be/r3GGV2TG_vw

Exercise 1 : ex1.java

- ◆ Example of Thread Creation and Execution (sleep() and join()).
- ◆ Try understanding, compiling, and executing the following code.

```
class C extends Thread {
    int i;
    C(int i) { this.i = i; }
    public void run() {
        System.out.println("Thread " + i + " says hi");
        try {
            sleep(500);
        } catch (InterruptedException e) {}
        System.out.println("Thread " + i + " says bye");
    }
}

class ex1 {
    private static final int NUM_THREAD = 10;
    public static void main(String[] args) {
        System.out.println("main thread start!");
        C[] c = new C[NUM_THREAD];
        for(int i=0; i < NUM_THREAD; ++i) {
            c[i] = new C(i);
            c[i].start();
        }
        System.out.println("main thread calls join()!");
        for(int i=0; i < NUM_THREAD; ++i) {
            try {
                c[i].join();
            } catch (InterruptedException e) {}
        }
        System.out.println("main thread ends!");
    }
}
```

Exercise 2 : parallel summation

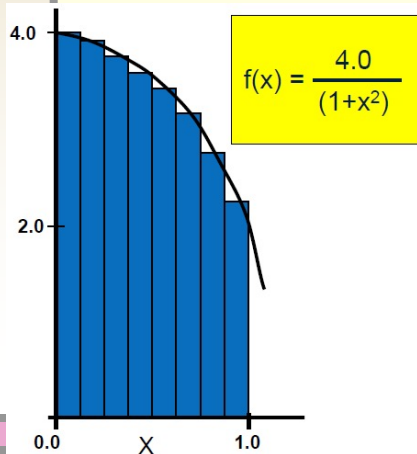
- ◆ Write a single-threaded and multi-threaded java program that computes the sum of integer numbers 1,2,...,NUM_END where NUM_THREAD is the number of threads.

```
class SumThread extends Thread {
    int lo, hi; // fields for communicating inputs
    int[] arr;
    int ans = 0; // for communicating result
    SumThread(int[] a, int l, int h) {
        lo=l; hi=h; arr=a;
    }
    public void run() {
        // insert your code here
    }
}

class ex2 {
    private static int NUM_END = 10000;
    private static int NUM_THREAD = 4; // assume NUM_END is divisible by NUM_THREAD
    public static void main(String[] args) {
        if (args.length==2) {
            NUM_THREAD = Integer.parseInt(args[0]);
            NUM_END = Integer.parseInt(args[1]);
        }
        int[] int_arr = new int [NUM_END];
        int i,s;
        for (i=0;i<NUM_END;i++) int_arr[i]=i+1;
        s=sum(int_arr);
        System.out.println("sum=" + s) ;
    }
    static int sum(int[] arr) {
        // insert your code here
    }
}
```

Exercise 3 : parallel integration

- ◆ Write a single threaded and multithreaded java programs that compute following numerical integration where the variable NUM_THREAD is # of threads and the variable NUM_STEP is # of steps.
- ◆ See the C/MPI codes below and rewrite them with JAVA language.



```
static long num_steps = 100000;

void main()
{
    int i;
    double pi, x, step, sum = 0.0;

    step = 1.0 / (double) num_steps;
    for (i = 0; i < num_steps; i++){
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x*x);
    }

    pi = step * sum;
    printf("Pi = %f\n", pi);
}
```

```
static long num_steps = 100000;

void main(int argc, char* argv[])
{
    int i_start, i_end, i, myid, numprocs;
    double pi, mypi, x, step, sum = 0.0;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    MPI_BCAST(&num_steps, 1, MPI_INT, 0, MPI_COMM_WORLD);

    i_start = my_id * (num_steps/numprocs)
    i_end = i_start + (num_steps/numprocs)

    step = 1.0 / (double) num_steps;
    for (i = i_start; i < i_end; i++) {
        x = (i + 0.5) * step
        sum = sum + 4.0 / (1.0 + x*x);
    }
    mypi = step * sum;

    MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (myid == 0)
        printf("Pi = %f\n", pi);

    MPI_Finalize();
}
```

Exercise 3 : parallel integration

```
// ex3_serial.java    (single threaded)
```

```
// ex3.java          (multithreaded)
```